CS 301
High-Performance Computing

Lab Report-08

Parallel Interpolation and Reverse Interpolation Using MPI and OpenMP hybrid Performance Analysis

Soham Mevada (202301484)
Atik Vohra (202301447)

Contents

1 Introduction

2 Hardware Details

2.1 LAB207 PCs

- Architecture: x86_64
- CPU(s): 12
- Threads per core: 2
- Cores per socket: 6
- Processor: Intel Core i5-12500
- L1 Cache: 288K
- L2 Cache: 7.5M
- L3 Cache: 18M

2.2 HPC Cluster

- Architecture: x86_64
- CPU(s): 16
- Threads per core: 1
- Cores per socket: 8
- Sockets: 2
- Processor: Intel Xeon E5-2640 v3
- L1 Cache: 32K
- L2 Cache: 256K
- L3 Cache: 20480K

3 Answers to Analysis and Questions in Lab 8

3.1 Q1. Pseudocode and Parallel Implementation

3.1.1 Pseudocode

```
Initialize MPI with MPI_THREAD_FUNNELED
Get rank and total number of MPI processes

Read metadata (NX, NY, NUM_Points_Global, Maxiter) on rank 0
```

```
Broadcast metadata to all MPI processes

Compute local partition:
    each MPI process gets a subset of particles

Read only local particles from input file
Initialize all particles as active (is_void = false)

Allocate:
    local_mesh
    global_mesh

Start timer

for iter = 1 to Maxiter do

    Reset local_mesh to zero

    // Interpolation Phase
    Parallel for each OpenMP thread:
        Create thread-local mesh buffer

        Parallel for each local particle:
            Skip if particle is void
            Compute grid cell containing particle
            Compute bilinear interpolation weights
            Accumulate weights into thread-local mesh

        Parallel reduction:
            Sum all thread-local meshes into process local_mesh

    // Communication Phase
    MPI_Allreduce(local_mesh, global_mesh, SUM)

    Compute global minimum and maximum mesh values

    // Mover Phase
    Parallel for each local particle:
        Skip if particle is void
        Interpolate force from neighboring grid cells
        Update particle position
        Mark particle void if it exits domain

end for
```

```
Stop timer

Rank 0:
    Save final mesh
    Print wall time, compute time, and communication time

Finalize MPI
```

3.1.2 Implementation Approach

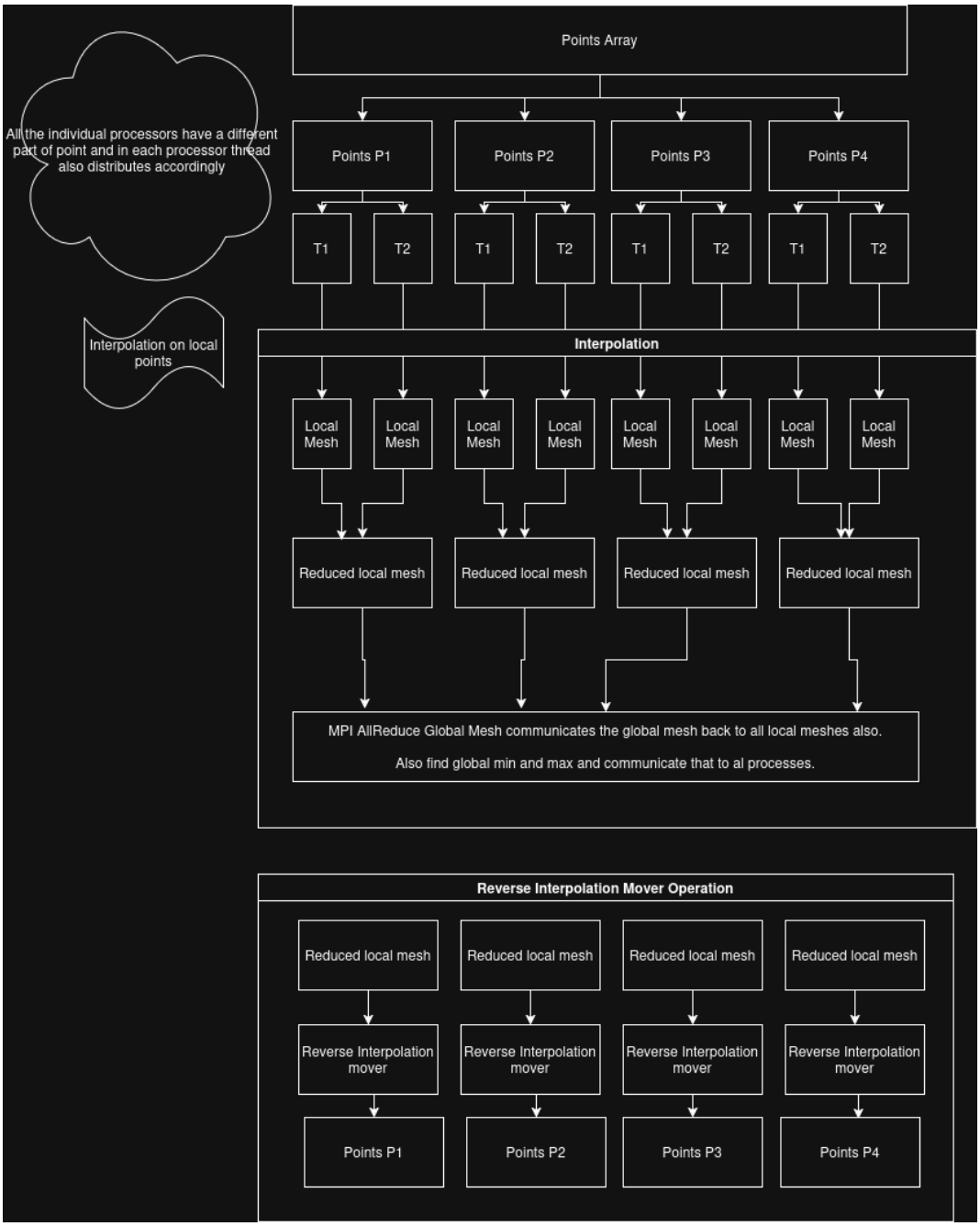

Figure 1: Illustrative diagram of the hybrid MPI + OpenMP implementation workflow. Each MPI process handles a subset of particles, performs local interpolation using thread-private buffers, synchronizes using MPI_Allreduce, and then executes the mover phase independently.

A hybrid MPI + OpenMP parallelization strategy was used to exploit both distributed-memory and shared-memory parallelism.

The global particle set is first partitioned across MPI processes, where each process is assigned a contiguous subset of particles. Each MPI process independently reads only its assigned particle block from the binary input file, reducing memory overhead and avoiding centralized particle storage.

Within each MPI process, OpenMP is used to parallelize both interpolation and mover phases.

During the interpolation phase, each thread processes a subset of particles and computes bilinear interpolation contributions to neighboring grid cells. Since multiple particles may contribute to the same grid location, directly updating the shared mesh would cause race conditions. To avoid this, each OpenMP thread maintains its own private mesh buffer.

After all particles are processed, a parallel reduction combines all thread-local meshes into a single process-local mesh.

Once each MPI process computes its local mesh, `MPI_Allreduce` is used to sum all local meshes into a global mesh. This ensures every process has an identical updated mesh before proceeding to the mover phase.

In the mover phase, particles are updated independently using interpolated force values computed from the global mesh. Since each particle is owned by only one MPI process and updated independently, this phase is embarrassingly parallel.

The implementation also computes global minimum and maximum mesh values for normalization using MPI reductions.

3.2 Q2. Race Condition Handling

Race conditions primarily arise during the interpolation phase because multiple particles may contribute to the same mesh cell simultaneously.

A naive shared-memory implementation would require atomic operations or critical sections for every mesh update, which would introduce significant synchronization overhead.

To eliminate this issue, a thread-private reduction strategy was implemented.

Each OpenMP thread allocates its own private mesh buffer:

```
double* t_mesh = &pad[tid * ms];
```

Each thread writes interpolation values only to its private buffer, ensuring no concurrent writes occur.

After all threads finish interpolation, a second parallel loop performs reduction:

```
for each mesh cell:
    sum contributions from all thread-local buffers
```

This completely avoids atomics and lock contention while preserving correctness.

Race conditions in the mover phase are naturally avoided because each particle is updated independently by only one thread.

MPI-level race conditions are avoided because processes do not share memory. Synchronization between processes is handled explicitly using:

```
MPI_Allreduce()
```

which safely aggregates local meshes into a global mesh.

Thus, race conditions were handled using:

- Thread-local mesh buffers for interpolation

- Parallel reduction for combining thread results
- Independent particle ownership across MPI processes
- MPI collective communication for global synchronization

3.3 Q3. Performance Analysis Across Input Configurations

3.3.1 Experimental Setup

All experiments were performed on the compute nodes (`gics1-gics4`) instead of the login node.

A login node is primarily intended for lightweight tasks such as code editing, compilation, file management, and job submission. Running computationally intensive applications on login nodes is discouraged because multiple users share the same resource.

In contrast, compute nodes are dedicated for execution of parallel workloads and provide isolated access to CPU resources, making them suitable for benchmarking and performance analysis.

The hybrid MPI + OpenMP implementation was executed for five input configurations. Performance was measured for total core counts ranging from 1 to 64, increasing by powers of two:

$$1, 2, 4, 8, 16, 32, 64$$

For each configuration, the following plots were generated:

- Speedup vs Total Cores
- Execution Time vs Total Cores
- Execution Time Heatmap (MPI Processes vs OpenMP Threads)

3.3.2 Generated Performance Plots

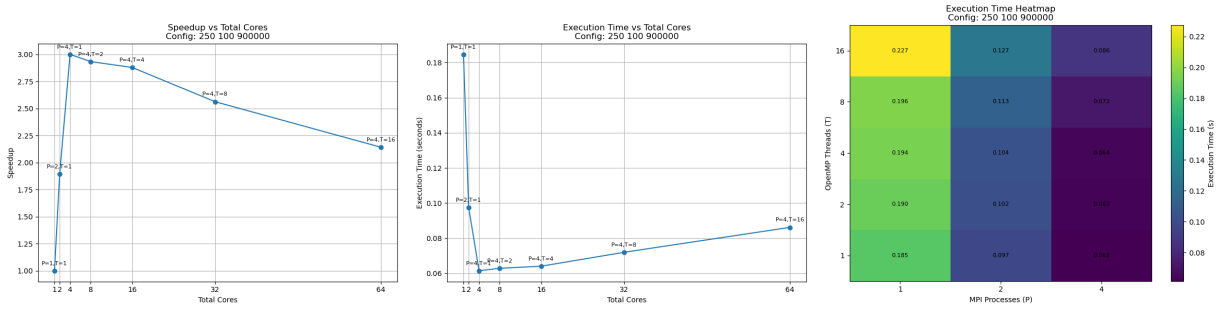

Figure 2: Performance plots for Configuration 1

Configuration 1

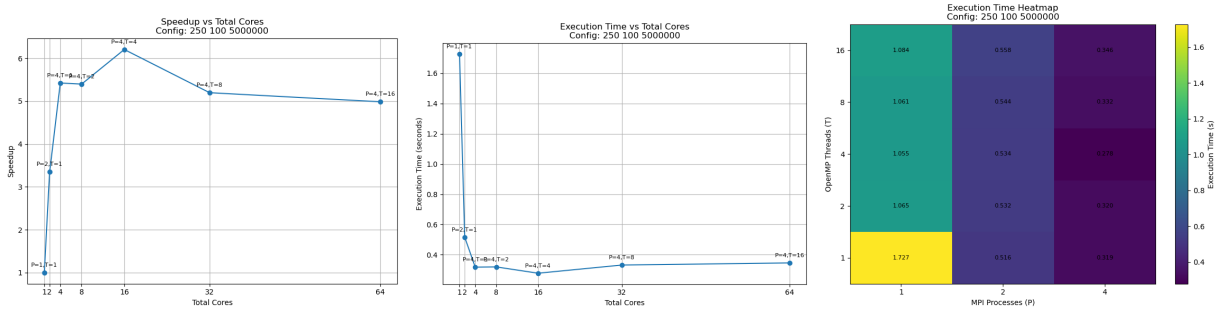

Figure 3: Performance plots for Configuration 2

Configuration 2

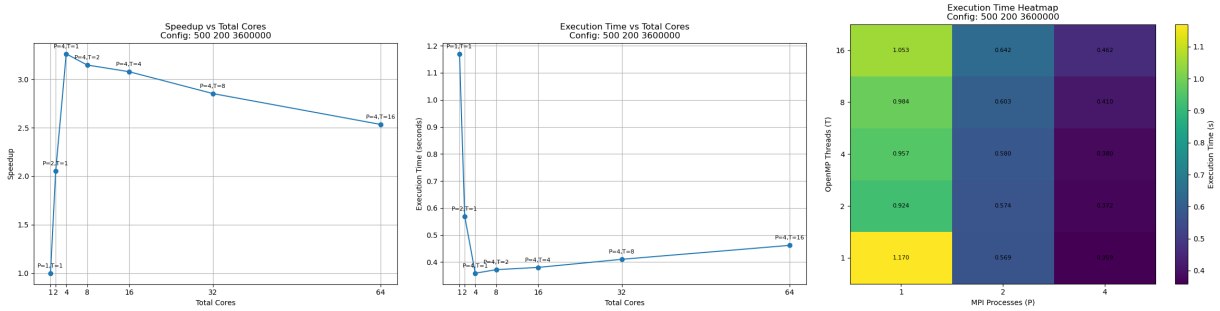

Figure 4: Performance plots for Configuration 3

Configuration 3

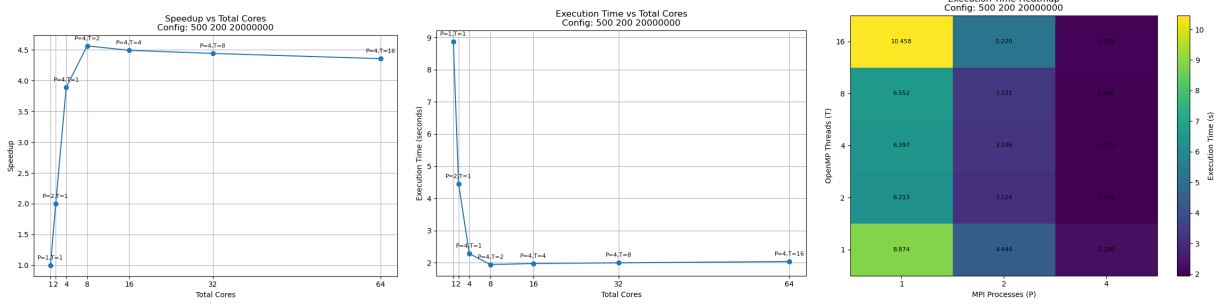

Figure 5: Performance plots for Configuration 4

Configuration 4

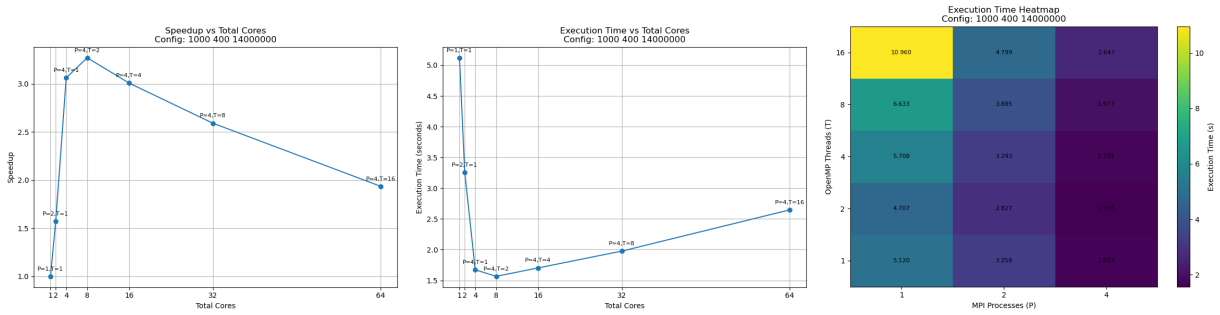

Figure 6: Performance plots for Configuration 5

Configuration 5

3.3.3 Observations

From the speedup plots, performance improves significantly when increasing cores from 1 to moderate core counts, indicating effective exploitation of both MPI and OpenMP parallelism.

However, beyond a certain number of cores, speedup growth begins to saturate. This is caused by:

- Increased MPI communication overhead due to frequent `MPI_Allreduce` operations
- Thread synchronization overhead during interpolation mesh reduction
- Reduced computational work per core at high parallelism levels

Execution time plots show rapid reduction in runtime initially, followed by diminishing returns as core count increases.

The heatmaps reveal that configurations using higher MPI process counts with moderate OpenMP thread counts generally outperform configurations relying heavily on threading alone.

Across most workloads, optimal performance is typically observed near:

- Moderate MPI process counts
- Low to moderate OpenMP thread counts

This indicates that distributed-memory parallelism contributes more effectively to scaling than excessive thread parallelism for this workload.

3.4 Q4. Parallel Efficiency and Scalability Analysis

Parallel efficiency is computed using:

$$Efficiency = \frac{Speedup}{Number\ of\ Cores}$$

where speedup is defined as:

$$Speedup = \frac{T_1}{T_p}$$

with T_1 being the serial execution time and T_p being execution time on p cores.

3.4.1 Computed Parallel Efficiency

Cores	Config 1	Config 2	Config 3	Config 4	Config 5
1	1.0000	1.0000	1.0000	1.0000	1.0000
2	0.9467	1.6738	1.0278	0.9985	0.7858
4	0.7499	1.3558	0.8151	0.9730	0.7651
8	0.3666	0.6747	0.3932	0.5706	0.4086
16	0.1799	0.3878	0.1923	0.2808	0.1879
32	0.0801	0.1624	0.0891	0.1388	0.0809
64	0.0335	0.0779	0.0396	0.0681	0.0302

Table 1: Parallel efficiency across all five input configurations

3.4.2 Observations

From the results, efficiency is highest at low core counts and decreases steadily as the number of cores increases.

Configurations 1, 3, 4, and 5 exhibit near-linear scaling up to approximately 4–8 cores, after which efficiency drops significantly.

For example:

- Configuration 1 drops from 0.9467 at 2 cores to only 0.0335 at 64 cores.
- Configuration 4 maintains the best scalability, retaining 0.5706 efficiency at 8 cores and 0.0681 at 64 cores.
- Configuration 5 shows the weakest large-scale performance, dropping to 0.0302 at 64 cores.

Configurations 2 and 3 show efficiency values greater than 1 for small core counts. This corresponds to superlinear speedup, which can occur due to:

- Better cache utilization when data is partitioned across cores
- Reduced memory contention
- Lower working set size per process
- Measurement variability for smaller workloads

Although superlinear behavior is theoretically uncommon, it is often observed in practice for memory-bound workloads.

3.4.3 Scalability Analysis

The implementation demonstrates good strong scaling at low to moderate core counts, particularly up to 4 or 8 cores.

Beyond this point, scalability degrades due to multiple overheads:

- Frequent `MPI_Allreduce` communication after every iteration
- Synchronization overhead during thread-local mesh reduction

- Reduced workload per core as total parallelism increases
- Increased OpenMP thread management overhead at high thread counts

The interpolation phase is particularly communication-sensitive because all processes must synchronize after constructing local meshes.

As the number of cores increases, communication and synchronization costs begin to dominate useful computation, limiting further speedup.

Therefore, while the hybrid implementation scales efficiently for moderate parallelism, it does not scale ideally to very high core counts.

Overall, the best practical operating region for this implementation is:

$$4 \leq \text{cores} \leq 16$$

where a balance is achieved between parallel acceleration and overhead costs.

3.5 Q5. Comparison with Serial Implementation

The performance of the hybrid MPI + OpenMP implementation was compared against the serial implementation using execution time at 1 core as baseline.

Speedup was computed as:

$$Speedup = \frac{T_1}{T_p}$$

where:

- T_1 is execution time on 1 core
- T_p is execution time on p cores

3.5.1 Maximum Speedup Achieved

The maximum speedup observed across all five configurations was:

$$6.2052$$

This was achieved for configuration:

$$250 \quad 100 \quad 5000000$$

at:

$$16 \text{ cores}$$

The corresponding optimal decomposition was:

$$P = 4, \quad T = 4$$

This indicates that a balanced hybrid decomposition using both MPI processes and OpenMP threads produced the best performance.

A summary of maximum speedups is shown below:

Configuration	Maximum Speedup	Core Count
250 100 900000	2.9996	4
250 100 5000000	6.2052	16
500 200 3600000	3.2605	4
500 200 20000000	4.5649	8
1000 400 14000000	3.2691	8

Table 2: Maximum speedup achieved for each configuration

The hybrid implementation consistently outperformed the serial implementation for all workloads, demonstrating clear benefits of parallelization.

3.6 Q6. Explanation of Speedup and Observations

The achieved speedup can be explained using standard HPC performance principles.

3.6.1 Reasons for Performance Improvement

The implementation benefits from two levels of parallelism:

- **MPI parallelism**: distributes particles across processes, reducing workload per process.
- **OpenMP parallelism**: parallelizes particle computations within each process.

Since particle interpolation and movement are largely independent operations, the workload is naturally data-parallel.

This allows efficient parallel execution during:

- interpolation phase
- mover phase

Additionally, thread-local mesh buffers eliminate race conditions without requiring expensive atomic operations.

3.6.2 Reasons for Limited Speedup

Despite initial performance gains, speedup saturates beyond moderate core counts.

This is due to:

- MPI communication overhead from repeated `MPI_Allreduce`
- OpenMP synchronization during mesh reduction
- Increasing thread management overhead
- Smaller computation per core at higher parallelism

As more cores are added, the communication-to-computation ratio increases, reducing scalability.

3.6.3 Effect of Problem Size

Larger workloads generally achieved better speedup.

This is because larger particle counts increase arithmetic intensity, making communication overhead relatively less significant.

For example, configuration:

$$250 \quad 100 \quad 5000000$$

achieved the highest speedup of 6.2052 because it contains sufficient computational work to amortize synchronization and communication costs.

Smaller workloads such as:

$$250 \quad 100 \quad 900000$$

show limited speedup because communication overhead dominates sooner.

3.6.4 Hybrid Parallelism Observations

Heatmap analysis showed that configurations with:

- higher MPI process counts
- moderate OpenMP thread counts

consistently performed better than configurations using excessive threading.

This suggests that for this workload:

- distributed-memory parallelism is more beneficial than aggressive shared-memory parallelism

because MPI reduces per-process working set size while avoiding excessive thread contention.

Overall, the implementation demonstrates effective hybrid parallelism, but scalability is ultimately constrained by communication and synchronization overheads.

3.7 Q7. Further Optimization with Unlimited HPC Resources

If unlimited HPC resources were available, several optimizations could further improve performance.

- **Increase MPI process count with domain decomposition:** Instead of assigning only particle subsets, the spatial mesh could also be partitioned across MPI processes. This would reduce memory footprint per process and improve cache locality.
- **Use non-blocking communication:** Replace blocking collective operations with:

  ```
  MPI_Iallreduce()
  ```

  to overlap communication with computation.
- **NUMA-aware memory placement:** On large multi-socket systems, allocating thread-local memory close to the executing core would reduce memory access latency.

- **GPU acceleration:** Both interpolation and mover phases are highly data-parallel and could be offloaded to GPUs using CUDA or OpenACC.
- **Hierarchical reduction:** Instead of reducing all thread-local meshes directly, perform:
  - intra-node reduction
  - inter-node MPI reduction

  to reduce communication overhead.

3.8 Q8. Load Imbalance and Performance Bottlenecks

Some load imbalance and bottlenecks were observed during execution.

3.8.1 Load Imbalance

Particles are initially partitioned equally across MPI processes.
However, as the simulation progresses:

- particles may move unevenly
- some particles become void after leaving the domain

This causes some processes to have fewer active particles than others, resulting in imbalance.
Consequently:

- some MPI processes finish computation earlier
- faster processes wait at synchronization points

particularly during:

`MPI_Allreduce()`

3.8.2 Performance Bottlenecks

The primary bottlenecks observed are:

- **Global communication overhead:** frequent `MPI_Allreduce` after each iteration
- **Interpolation reduction overhead:** combining thread-local meshes
- **Memory bandwidth pressure:** large mesh arrays accessed repeatedly

At high core counts, communication and synchronization overhead dominate useful computation, limiting scalability.

3.9 Q9. Alternative Approaches for Performance Improvement

One possible alternative approach is to replace dense mesh storage with a sparse representation.
Currently, each thread maintains a full private mesh buffer, which increases:

- memory usage
- cache pressure
- reduction cost

Instead, each thread could maintain only updated cells using sparse structures such as:

- hash maps
- coordinate lists
- compressed sparse row (CSR) format

Advantages:

- lower memory overhead
- fewer reduction operations
- better cache efficiency

Another possible improvement is dynamic OpenMP scheduling:

```
#pragma omp parallel for schedule(dynamic)
```

which can reduce imbalance when particle workloads become uneven.

3.10 Q10. Alternative OpenMP-MPI/Data Structure Optimizations

To further improve performance, the following alternative approaches can be explored.

3.10.1 Spatial Domain Decomposition

Instead of particle-based decomposition, divide the simulation mesh spatially among MPI processes.
Advantages:

- better locality
- smaller mesh per process
- reduced communication volume

This may require exchanging boundary particles between neighboring processes.

3.10.2 Particle Binning

Particles can be grouped into bins based on spatial cell location.

Benefits:

- faster neighborhood lookup
- improved cache locality
- reduced interpolation overhead

This is particularly beneficial when particle count is very large.

3.10.3 Two-Level Reduction Strategy

Instead of reducing all thread-local meshes directly into a full mesh:

1. locally merge thread-private contributions
2. communicate only merged data across MPI processes

This reduces:

- synchronization cost
- communication overhead

3.10.4 Hybrid Tuning

Experimental results suggest best performance is achieved using:

- moderate MPI process counts
- moderate OpenMP thread counts

Future tuning can automatically search for optimal:

$$(P, T)$$

combinations using autotuning frameworks.

This would improve portability across different HPC architectures.

3.11 Q11. Analysis of Interpolation and Mover Phases

To identify the computational bottleneck, execution times of interpolation and mover phases were measured separately.

The analysis was performed on:

$$1000 \quad 400 \quad 14000000$$

using:

$$P = 4, \quad T = 1$$

Measured timings are shown below:

Phase	Time (s)
Interpolation	1.650012
Mover	2.111076
Communication	0.053000
Wall Time	3.820875

Table 3: Phase-wise execution time analysis for Configuration 5

3.11.1 Bottleneck Analysis

From the measurements, the mover phase consumes the highest execution time:

$$2.111076 > 1.650012$$

Thus, the mover phase is the primary bottleneck for this configuration.
The mover phase is computationally expensive because each particle performs:

- interpolation of neighboring mesh values
- force computation
- position updates
- boundary checking and invalidation

Unlike interpolation, which mainly performs weighted accumulation into mesh cells, mover performs multiple floating-point computations per particle and updates particle state.
Since configuration 5 contains 14 million particles, mover workload becomes dominant.

3.11.2 Communication Overhead

Communication overhead is relatively small:

$$0.053000 \text{ s}$$

compared to both interpolation and mover phases.
This indicates that for this configuration:

- computation dominates communication
- MPI communication is not the primary performance limiter

Therefore, optimizing computational kernels is likely to yield greater benefits than communication optimization.

3.11.3 Optimization Implications

Since mover is the bottleneck, future optimization efforts should focus on:

- improving cache locality for particle access
- vectorization of particle update computations
- reducing branch overhead during boundary checks
- improving memory layout for particle structures

Potential improvements include:

- Structure of Arrays (SoA) layout instead of Array of Structures (AoS)
- SIMD vectorization
- particle blocking or spatial binning

4 Conclusion

In this assignment, a hybrid MPI + OpenMP implementation was developed for particle interpolation and movement simulation.

The implementation uses:

- MPI for distributed-memory parallelism
- OpenMP for shared-memory parallelism
- thread-local mesh buffers to avoid race conditions

Experimental evaluation across five input configurations demonstrated clear speedup over the serial implementation.

The maximum speedup achieved was:

$$6.2052$$

for configuration:

$$250 \quad 100 \quad 5000000$$

using 16 cores.

Performance analysis showed that:

- the implementation scales well up to moderate core counts
- efficiency decreases at high core counts due to communication and synchronization overhead
- hybrid configurations with moderate MPI process counts and thread counts perform best

Phase-wise profiling showed that for large workloads, the mover phase is the dominant computational bottleneck.

Overall, the hybrid implementation successfully improves execution time while maintaining correctness and demonstrates practical application of MPI, OpenMP, synchronization avoidance, and performance analysis techniques in high-performance computing.